
AWS WAF Security Intelligence & Observability Platform

Service-by-Service Architecture Guide, Interview Preparation
& Project Presentation Playbook

Single-account sandbox deployment	us-west-2	Terraform-provisioned
-----------------------------------	-----------	-----------------------

Dimension	What this project demonstrates
Security Engineering	WAF rule design, managed rule groups, rate-based rules, log-driven threat analysis
Data Engineering	Streaming ingestion, partitioned data lake on S3, schema-on-read with Glue + Athena
Serverless / Automation	Event-driven scheduling, Lambda report generation, pub/sub notification via SNS
Observability / SRE	Metrics pipelines, PromQL alert rules, multi-dashboard Grafana design, alert routing
Platform / IaC	Modular Terraform, multi-environment design, IAM least privilege, KMS encryption-by-default

Prepared for portfolio & interview use

Table of Contents

- 1. Executive Summary**
- 2. Architecture Diagram**
- 3. Service-by-Service Deep Dive**
 - 3.1. Edge & Traffic Layer — WAF, ALB, VPC, Internet Gateway
 - 3.2. Log Ingestion & Analytics — Firehose, S3, Glue, Athena
 - 3.3. Scheduled Reporting & Alerting — EventBridge, Lambda, SNS
 - 3.4. Metrics & Dashboards — CloudWatch, EC2, Prometheus, Grafana, Alertmanager
 - 3.5. Security & Governance — KMS, IAM
- 4. End-to-End Request & Data Flow Walkthrough**
- 5. Prometheus Monitoring Internals**
- 6. Grafana Dashboards**
- 7. Testing Strategy**
- 8. How to Present This as a Project**
- 9. Interview Questions & Model Answers**
 - 9.1–9.8. Subsystem Q&A; sections
- 10. Quick-Reference Cheat Sheet**

1. Executive Summary

This platform is a single-AWS-account security and observability system built around AWS WAF. It does three things end to end: **(1)** it stops malicious traffic — SQL injection, cross-site scripting, bot abuse, and request-rate floods — at the edge before it reaches the application; **(2)** it captures every WAF decision as a structured log and turns those logs into queryable, reportable security intelligence using a serverless analytics pipeline; and **(3)** it gives engineers a live, self-hosted observability stack — Prometheus, Grafana, and Alertmanager — so that both security events and platform health are visible in real time, not just after the fact in the AWS console.

The project intentionally combines four distinct AWS skill areas that DevOps and Platform Engineer interviews probe separately: **edge security** (WAF), **serverless data engineering** (Firehose → S3 → Glue → Athena), **event-driven automation** (EventBridge → Lambda → SNS), and **self-managed observability** on EC2 (Prometheus / Grafana / Alertmanager scraping CloudWatch via an exporter). All of it is provisioned through versioned, modular Terraform.

What makes this project worth discussing in an interview

- It is **not a single service demo** — it is a system with five cooperating subsystems, which mirrors how real platform teams design security + observability stacks.
- It **deliberately avoids being 'just CloudWatch'** — running Prometheus and Grafana on EC2 alongside CloudWatch shows you understand both the AWS-native and the open-source / self-hosted observability worlds.
- It has a **real data pipeline** (Firehose → S3 → Glue → Athena) doing log analytics, not just metrics — a frequently underestimated but high-value skill combination.
- It is **fully Terraform-defined** with modules, multiple environments, and automated tests — so you can speak to Infrastructure as Code maturity, not just 'I clicked around in the console.'

2. Architecture Diagram

The diagram below represents the complete platform layout across four visual zones inside the AWS Cloud boundary. Reading left to right roughly follows the direction traffic and data travel. The companion file **aws-architecture.drawio** can be opened in diagrams.net for interactive exploration.

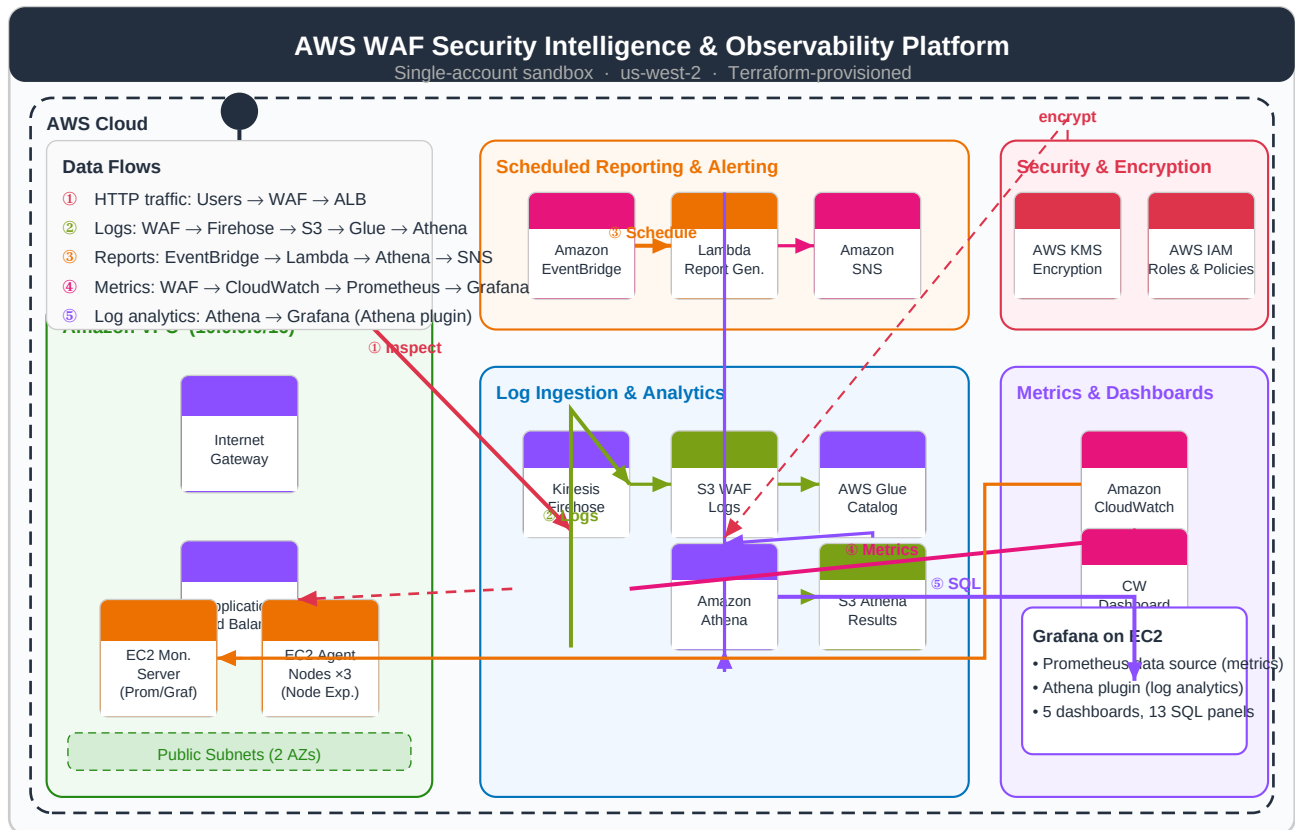


Figure 1: AWS WAF Security Intelligence & Observability Platform — vector architecture diagram (reproduced from *aws-architecture.drawio*).

Diagram Zones

VPC (green)	Internet Gateway, ALB, EC2 Monitoring Server, EC2 Agent Nodes ×3	Everything that needs a network path — public subnets across 2 AZs
Log Ingestion & Analytics (blue)	Kinesis Firehose, S3 (WAF logs), Glue, Athena, S3 (Athena results)	Turns raw WAF log records into a queryable data lake
Scheduled Reporting & Alerting (orange)	EventBridge, Lambda, SNS	Cron-driven report generation and email delivery
Metrics & Dashboards (purple)	CloudWatch, CloudWatch Dashboard, Grafana (on monitoring EC2)	Real-time metrics through both native AWS dashboards and self-hosted Grafana
Security & Encryption (red)	KMS, IAM	Cross-cutting controls applied to every other service

The Five Numbered Data Flows

①	Inspect	Users → AWS WAF Web ACL → Application Load Balancer. WAF evaluates every request against its rules before the ALB ever forwards it to the application.
②	Logs	WAF → Kinesis Firehose → S3 → Glue → Athena. Every allow/block decision becomes a structured, partitioned log record available for SQL analysis within minutes.
③	Schedule	EventBridge → Lambda → Athena → SNS. Cron rules trigger a Lambda that queries Athena and emails an HTML/CSV report.
④	Metrics	WAF → CloudWatch → EC2 (CloudWatch Exporter) → Prometheus → Grafana. AWS-native metrics are pulled into the self-hosted stack for unified dashboards and alerts.
⑤	Log analytics	Athena → Grafana (Athena data source). Grafana queries Athena directly for log-driven panels (top attackers, countries, URIs) alongside Prometheus-backed metric panels.

3. Service-by-Service Deep Dive

Each service follows the same structure: what it is, its exact role in this platform, key configuration choices, and why it was chosen over the alternatives. This is the level of detail interviewers expect when they ask 'walk me through this box on your diagram.'

3.1 Edge & Traffic Layer

AWS WAF (Web Application Firewall) · SECURITY

Role in this platform: The platform's first line of defense, attached as a Web ACL directly to the Application Load Balancer. Every request is evaluated against AWS Managed Rule Groups (SQLi injection, known bad inputs, common exploits) and a rate-based rule, and every decision is emitted as a log record.

Key configuration choices:

- AWS Managed Rule Groups: SQLiRuleSet, CommonRuleSet (XSS/bad inputs), and a bot-control rule group.
- A rate-based rule blocks any single IP exceeding a configured request threshold within a 5-minute window.
- Logging enabled via native WAF → Kinesis Firehose log destination (not CloudWatch Logs) for direct S3 delivery with partitioning.
- Web ACL is regional (not CloudFront/global) because the ALB sits in a single region for this sandbox deployment.

Why this, not an alternative: WAF integrates natively with ALB/CloudFront, requires no extra compute, and its managed rule groups are maintained by AWS threat intelligence teams — building equivalent SQLi/XSS detection yourself would mean operating and patching that detection logic yourself.

Application Load Balancer (ALB) · NETWORKING

Role in this platform: Sits behind the WAF Web ACL as the single entry point for application traffic inside the VPC. The WAF association happens at this layer, and the ALB itself emits CloudWatch metrics (RequestCount, 5XX) that feed the observability stack.

Key configuration choices:

- Deployed across two public subnets in two Availability Zones for basic high availability.
- Access logs can be enabled to S3 independently of WAF logs.
- Health checks against backend targets drive the Blackbox Exporter probe target used in the Prometheus stack.

Why this, not an alternative: ALB rather than a Network Load Balancer (NLB) because WAF only attaches to layer-7 resources — an NLB operates at layer 4 and cannot host a WAF Web ACL at all.

Amazon VPC · NETWORKING

Role in this platform: Provides network isolation for the ALB and the EC2 monitoring fleet. Public subnets (spanning two AZs) host internet-facing resources; the Internet Gateway is the sole path in and out.

Key configuration choices:

- CIDR block 10.0.0.0/16 with public subnets sized to comfortably fit the ALB ENIs and four EC2 instances.
- Security groups scope inbound access tightly — only 80/443 from the internet to the ALB.
- No NAT Gateway in the minimal sandbox layout (cost-optimised for a portfolio deployment).

Why this, not an alternative: A VPC is non-negotiable for anything beyond the AWS default network — it's what makes security-group scoping, multi-AZ placement, and later a private-subnet hardening pass possible.

3.2 Log Ingestion & Analytics

Amazon Kinesis Data Firehose · STREAMING

Role in this platform: Receives the WAF log stream directly and writes it to the S3 WAF-logs bucket in GZIP-compressed, year/month/day-partitioned objects.

Key configuration choices:

- Buffering configured on a size/time hint (5 MB or 300 seconds, whichever comes first).
- Dynamic partitioning by year/month/day/hour prefixes so Athena and Glue can prune partitions instead of scanning the entire bucket.
- GZIP compression reduces both S3 storage cost and the bytes Athena scans per query (Athena bills by data scanned).
- Delivery failures retry automatically and land in an error-output S3 prefix rather than silently dropping records.

Why this, not an alternative: Firehose over a custom Lambda-based log shipper because it's fully managed, natively supported as a WAF logging destination, and handles backpressure and retries that you would otherwise have to build yourself.

Amazon S3 (WAF Logs + Athena Results Buckets) · STORAGE

Role in this platform: The waf-logs/ prefix is the landing zone for every Firehose delivery and the table location the Glue Catalog points at. A separate Athena-results bucket holds query output.

Key configuration choices:

- Lifecycle rules transition older log partitions to Infrequent Access or Glacier after a retention window.
- Server-side encryption with a customer-managed KMS key (see Section 3.5).
- Bucket policies deny non-TLS access and restrict write access to the Firehose delivery role only.

Why this, not an alternative: S3 is the only sensible choice for this volume and access pattern — it's the cheapest durable store AWS offers, and both Athena and Glue are built to query S3 directly without a database engine running 24/7.

AWS Glue (Catalog & Crawler) · DATA CATALOG

Role in this platform: Crawls the waf-logs/ prefix on a schedule, infers the JSON log schema, and registers/updates the waf_logs table (including new date partitions) so Athena always has an up-to-date view without manual DDL.

Key configuration choices:

- Scheduled crawler run (e.g. hourly) rather than continuous — WAF log schema is stable and new data is mainly new partitions.
- Crawler IAM role is scoped to read-only on the source bucket and catalog-write only.
- Partition projection can be used as a lower-cost alternative once the date-partition pattern is fixed.

Why this, not an alternative: Glue over manually maintaining Athena DDL/partitions because the crawler keeps schema and partitions self-updating — critical when log volume and date range grow daily.

Amazon Athena · ANALYTICS

Role in this platform: The query layer for everything downstream: ad-hoc forensic queries, the 6 scheduled report queries run by Lambda, and the live log-analytics panels in Grafana.

Key configuration choices:

- Workgroup configured with a per-query data-scanned limit to guard against accidental full-table-scan cost.
- Query results bucket and encryption configured at the workgroup level.
- Queries are partition-aware (filter on year/month/day) to minimise bytes scanned.

Why this, not an alternative: Athena over Redshift or RDS because there's no cluster to manage and cost is pay-per-query — appropriate for a workload that's bursty (reports run a few times a day) rather than constantly hammered.

3.3 Scheduled Reporting & Alerting

Amazon EventBridge · EVENTING

Role in this platform: Drives three schedule rules — daily at 06:00 UTC, weekly Monday 07:00 UTC, and monthly on the 1st at 08:00 UTC — each invoking the Lambda Report Generator with a different report-period parameter.

Key configuration choices:

- Each schedule passes a JSON input (e.g. {"period": "daily"}) so a single Lambda handles all three cadences.
- Rules are defined in Terraform alongside the Lambda so schedule changes are code-reviewed, not console-edited.

Why this, not an alternative: EventBridge Scheduler over a cron daemon on EC2 because it needs zero compute, has built-in retry semantics, and removes an entire server you'd otherwise patch and monitor just to fire three cron jobs a month.

AWS Lambda (Report Generator) · COMPUTE

Role in this platform: On each EventBridge trigger, runs six parameterised Athena queries (top attackers, countries, SQLi, XSS, bot, and an overall summary), assembles results into HTML and CSV, writes them to S3, and publishes the report via SNS.

Key configuration choices:

- Athena queries are issued asynchronously (StartQueryExecution) and polled, since Lambda's own timeout can't block on a synchronous Athena call.
- IAM role grants only `athena:StartQueryExecution/GetQueryResults`, `s3:GetObject/PutObject` on specific buckets, and `sns:Publish`.
- Function timeout and memory are tuned based on actual report-generation duration observed in testing.

Why this, not an alternative: Lambda over a containerised script because the workload is short-lived and infrequent — paying for an always-on container for a few invocations a day would be wasteful.

Amazon SNS (Simple Notification Service) · MESSAGING

Role in this platform: Two roles: delivering scheduled HTML/CSV reports by email, and acting as the receiver for Prometheus Alertmanager's routed alerts.

Key configuration choices:

- Separate topics for reports vs. operational alerts, so recipients can subscribe to one without the other.
- Topic access policy restricts Publish to the specific Lambda and Alertmanager.
- SNS topics are KMS-encrypted at rest, consistent with the platform's encrypt-everything posture.

Why this, not an alternative: SNS over building a custom email-sending Lambda (e.g. with SES called directly) because fan-out to multiple subscriber types is a config change, not a code change, once SNS is the integration point.

3.4 Metrics & Dashboards

Amazon CloudWatch · OBSERVABILITY

Role in this platform: The source of AWS-side metrics for WAF (BlockedRequests/AllowedRequests), Firehose, Lambda, ALB, and Athena. A lightweight native CloudWatch Dashboard also exists for a quick AWS-console view.

Key configuration choices:

- WAF sample rules also publish custom CloudWatch metrics per rule, so block-rate can be broken down by which rule fired.
- CloudWatch Alarms exist as a secondary, AWS-native safety net even though Alertmanager is the primary alerting path.

Why this, not an alternative: CloudWatch isn't replaced — it's the unavoidable source of truth for AWS service metrics. The design choice is layering Prometheus on top via the exporter rather than building every alert natively in CloudWatch.

Amazon EC2 (Monitoring Server + Agent Nodes) · COMPUTE

Role in this platform: The monitoring server runs Prometheus, Grafana, Alertmanager, and the CloudWatch Exporter. Three agent nodes each run a Node Exporter, simulating a small fleet.

Key configuration choices:

- Security groups allow inbound 9090 (Prometheus), 3000 (Grafana), 9093 (Alertmanager) only from within the VPC.
- Prometheus, Grafana, and Alertmanager installed via a provisioning script (user-data) so the stack is reproducible.
- Sized as t3.small/t3.medium — portfolio-scale workload, not production traffic volume.

Why this, not an alternative: Self-managed EC2 + open-source Prometheus/Grafana rather than Amazon Managed Service specifically to demonstrate hands-on operational ownership — config files, retention tuning, alert rule authoring.

Prometheus · OBSERVABILITY

Role in this platform: The central metrics store for the whole platform. It scrapes 9 jobs: itself, three Node Exporters, local Node Exporter, CloudWatch Exporter, Blackbox Exporter, and AWS-specific filtered jobs.

Key configuration choices:

- 9 scrape jobs configured with relabeling so AWS CloudWatch dimension labels map cleanly to Prometheus label names.
- 15-second scrape interval and 15-second rule-evaluation interval.
- 30-day local TSDB retention — long enough for trend analysis, short enough to keep disk usage bounded.
- 12 alert rules spanning both security (block-rate, SQLi/XSS/bot surges) and infrastructure health.

Why this, not an alternative: Prometheus over relying purely on CloudWatch Alarms because PromQL gives far richer alerting expressions (rate(), multi-metric comparisons, for-duration conditions).

Grafana · OBSERVABILITY

Role in this platform: Runs on the monitoring EC2 instance with two data sources: Prometheus (metrics) and Athena (log analytics). Hosts five dashboards with 13 Athena SQL panel queries.

Key configuration choices:

- Provisioned as JSON dashboard definitions checked into the repo — dashboards are version-controlled like code.
- Each dashboard has a defined audience: Executive (roll-up), Security Overview and Threat Intelligence (operator-facing), Infrastructure, and WAF Log Analytics.
- Athena panels are time-boxed with template variables (date range) for daily review or monthly retrospective.

Why this, not an alternative: Grafana over CloudWatch Dashboards or Amazon Managed Grafana because it's the one place that can show a Prometheus metric panel and an Athena log-analytics panel side by side.

Alertmanager · OBSERVABILITY

Role in this platform: Receives all 12 alert rules' firing events from Prometheus and routes them by severity — critical alerts to a more urgent receiver path — ultimately publishing to SNS.

Key configuration choices:

- Routing tree groups alerts by alertname and severity label so one incident doesn't spam five separate emails.
- Inhibition rules suppress lower-priority alerts when a related critical alert is already firing.

Why this, not an alternative: Alertmanager is the natural pairing with Prometheus — it's the piece that turns 'a rule evaluated true' into 'the right person got one consolidated notification.'

3.5 Security & Governance

AWS KMS (Key Management Service) · SECURITY

Role in this platform: Provides the customer-managed key(s) used to encrypt the S3 WAF-logs bucket, the Athena-results bucket, and the SNS topics.

Key configuration choices:

- Key policy explicitly lists which roles (Firehose, Lambda, Athena workgroup) may use the key — no wildcard principal access.
- Key rotation enabled (annual automatic rotation) as a baseline hygiene control.
- Customer-managed keys mean key usage shows up distinctly in CloudTrail and the key can be disabled/deleted independently.

Why this, not an alternative: Customer-managed KMS keys over default SSE-S3 encryption because CMKs give explicit, auditable control over exactly which IAM principals can decrypt — a requirement most security reviews (SOC 2, PCI-DSS) ask for by name.

AWS IAM (Identity and Access Management) · SECURITY

Role in this platform: Defines a distinct, least-privilege role for every service boundary: Firehose delivery role, Lambda execution role, Glue crawler role, and EC2 instance profile for the monitoring server.

Key configuration choices:

- Every role is service-specific and resource-scoped (ARN-restricted), not account-wide.
- No IAM users with long-lived access keys — everything runs under roles assumed by the service itself.
- A test_terraform_structure.py unit test explicitly checks for the absence of hardcoded secrets in the Terraform source.

Why this, not an alternative: Fine-grained, per-service IAM roles instead of one broad shared role because it limits blast radius — if the Lambda function were ever compromised, its role cannot touch the Glue catalog, the EC2 fleet, or any other bucket.

4. End-to-End Request & Data Flow Walkthrough

This is the narrative you tell when someone says 'walk me through what happens when a malicious request hits this system.' It stitches every service into one chronological story.

1	Request arrives	A client request hits the Internet Gateway and is routed toward the Application Load Balancer.
2	WAF inspection	Before the ALB forwards anything, the associated WAF Web ACL evaluates the request against managed rule groups and the rate-based rule.
3	Decision + log	WAF makes a BLOCK or ALLOW/COUNT decision. Either way, a structured log record is emitted to Kinesis Firehose in the same instant.
4	Allowed traffic continues	If allowed, the ALB forwards the request to the application target group as normal; the application responds.
5	Log buffering & delivery	Firehose buffers the WAF log record and writes a GZIP-compressed, date-partitioned object to the S3 WAF-logs bucket.
6	Catalog update	On its next scheduled run, the Glue crawler discovers the new partition and updates the waf_logs table in the Glue Catalog.
7	Metrics in parallel	Independently, WAF also publishes BlockedRequests/AllowedRequests metrics to CloudWatch in near real time.
8	Exporter pull	The CloudWatch Exporter on the monitoring EC2 polls CloudWatch and re-exposes those metrics in Prometheus exposition format on :9106.
9	Scrape & evaluate	Prometheus scrapes the exporter every 15 seconds and evaluates its alert rules against the fresh data.
10	Alert routing	If a rule fires, Alertmanager groups/deduplicates it and routes it by severity to SNS, which emails the on-call recipient.
11	Dashboards update	Grafana's Prometheus-backed panels reflect new block-rate within one scrape interval; Athena-backed panels reflect new log data with a small additional lag.
12	Scheduled reporting	On its cron schedule, EventBridge triggers the Lambda Report Generator, which runs its six Athena queries and emails an HTML/CSV summary via SNS.

Key insight: There are two parallel paths from the same WAF decision — a **logs path** (Firehose → S3 → Glue → Athena, optimised for rich SQL analysis) and a **metrics path** (CloudWatch → Exporter → Prometheus, optimised for low-latency alerting). Reports and forensic dashboards read from the logs path; live alerting and real-time dashboards read from the metrics path. This separation — analytics vs. alerting on different pipelines tuned for their own latency needs — is a deliberate architectural decision, not an accident, and it's worth saying explicitly in an interview.

Prometheus Monitoring Internals

Interviewers will probe this section specifically. Knowing the exact scrape jobs, metric names, and alert thresholds — not just 'we used Prometheus' — is what separates a credible answer from a vague one.

5.1 Scrape Targets

Job	Target	Purpose
prometheus	localhost:9090	Self-monitoring
node-exporter	Agent EC2s x3, :9100	CPU, memory, disk, network
node-exporter-monitoring	localhost:9100	Monitoring server's own health
cloudwatch-exporter	localhost:9106	Re-exposed AWS WAF/Firehose/Lambda/ALB/Athena metrics
blackbox-exporter	ALB DNS via :9115	HTTP probe_success for ALB reachability
aws-waf-cloudwatch	filtered aws_wafv2_*	WAF-specific metric relabeling
aws-firehose-cloudwatch	filtered aws_firehose_*	Delivery pipeline health
aws-lambda-cloudwatch	filtered aws_lambda_*	Report generator errors
aws-athena-cloudwatch	filtered aws_athena_*	Query volume monitoring

5.2 Key Prometheus Metrics

`aws_wafv2_blocked_requests_sum` — Blocked requests, broken down by rule

`aws_wafv2_allowed_requests_sum` — Allowed requests, broken down by rule

`aws_firehose_delivery_to_s3_success_average` — Log delivery pipeline health

`aws_lambda_errors_sum` — Report-generator Lambda failures

`node_cpu_seconds_total` — Infrastructure CPU usage (counter; rate() applied)

`node_memory_MemAvailable_bytes` — Memory pressure across the EC2 fleet

`probe_success{job="blackbox-alb"}` — ALB reachability (1 = healthy, 0 = failed probe)

`up` — Prometheus built-in: whether a scrape target answered

5.3 Alert Rules (waf-alerts.yml)

Alert	Severity	Condition
HighBlockRate	Critical	Block rate exceeds 10 requests/sec, sustained for 5 minutes
SQLiSurge	High	SQL-injection rule blocks elevated above baseline
XSSSurge	High	XSS / bad-inputs rule blocks elevated above baseline
BotAttackSurge	Medium	Bot-control rule blocks elevated above baseline

Alert	Severity	Condition
TopAttackerSpike	High	Sudden coordinated attack pattern from one source
HighCPUUsage	Medium	Node CPU above 85% for 10 minutes
HighMemoryUsage	Medium	Memory usage above 90% for 10 minutes
DiskSpaceLow	High	Free disk below 15%
LambdaErrors	High	Report generator Lambda is failing
FirehoseDeliveryFailures	Critical	S3 delivery success rate below 95%
PrometheusTargetDown	Critical	A scrape target is unreachable
BlackboxProbeFailure	High	ALB health probe failed

5.4 Storage and Retention

- **TSDB retention:** 30 days
- **Scrape interval:** 15 seconds
- **Rule evaluation interval:** 15 seconds

6. Grafana Dashboards

Five dashboards, each with a distinct audience and data source mix — a detail worth naming explicitly, since 'we have dashboards' is weak and 'we designed dashboards by audience and data source' is strong.

Dashboard	Data Source	Purpose
WAF Security Overview	Prometheus	Block/allow rates, SQLi/XSS/Bot breakdowns, node CPU, ALB probe status — the operator's day-to-day view
Threat Intelligence	Prometheus	Attack trends segmented by rule type, over time
Executive Dashboard	Prometheus	Block-rate %, platform health summary — roll-up for non-technical stakeholders
Infrastructure Monitoring	Prometheus	CPU, memory, disk, Firehose health, scrape-target availability
WAF Log Analytics	Athena	Top attackers, attacker countries, targeted URIs, recent blocks — 13 SQL-backed panels

Reporting flow: EventBridge cron rules (daily 06:00 UTC, weekly Monday 07:00 UTC, monthly 1st 08:00 UTC) trigger the Lambda Report Generator → six Athena queries (top_attackers, countries, sqli, xss, bot, summary) → HTML and CSV output written to S3 reports/ → notify via SNS email. Independently, whenever a Prometheus alert fires, Alertmanager routes it by severity to SNS.

7. Testing Strategy

A test suite is one of the strongest signals of engineering maturity you can put in a portfolio project — most personal AWS projects have none. Being able to say 'here's how I validated my Terraform and my Lambda logic' sets this project apart immediately.

Run command:

```
pip install -e '.[dev]' && pytest tests/unit/ -v && bash tests/terraform/validate.sh dev
```

7.1 Unit Tests

[test_prometheus_alertmanager.py](#) — Validates Prometheus YAML structure, 30-day retention, all 12 alert rules, and Alertmanager routing config

[test_grafana_dashboards.py](#) — Validates Grafana JSON correctness, panel grid positioning, and Athena-dashboard SQL panel definitions

[test_dashboard_validation.py](#) — Validates CloudWatch and Grafana dashboard structure end to end

[test_lambda_report_generator.py](#) — Validates CSV/HTML report generation, Athena query placeholder substitution, and mocks the Lambda handler

[test_terraform_structure.py](#) — Validates module file layout, per-environment configs, and confirms no hardcoded secrets anywhere in Terraform source

7.2 Integration and End-to-End Tests

[test_infrastructure.py](#) — Checks live ALB response, confirms WAF protection is active, validates S3 config, checks Athena query files

[tests/e2e/run_e2e.py](#) — Full end-to-end validation after a real deployment

[tests/terraform/validate.sh](#) — Runs terraform fmt, validate, and plan for every environment

[tests/terraform/security_scan.sh](#) — Static security checks against the Terraform source

7.3 Coverage Areas

- Prometheus scrape configuration and all 12 alert rules across security and infrastructure
- 5 Grafana dashboards including the Athena log-analytics dashboard with 13 SQL targets
- Lambda report generator logic against all 6 Athena query templates
- Terraform modules: vpc, alb, waf, firehose, s3, glue, athena, lambda, monitoring, iam, kms, sns, cloudwatch

8. How to Present This as a Project

8.1 One-sentence framing

"I built a single-account AWS security and observability platform that puts AWS WAF in front of an ALB, streams every WAF decision through a serverless analytics pipeline (Firehose → S3 → Glue → Athena) for log forensics and scheduled reporting, and pairs it with a self-hosted Prometheus/Grafana/Alertmanager stack for real-time metrics and alerting — all provisioned through modular Terraform with an automated test suite."

8.2 Resume Bullet Points

- Designed and deployed a multi-service AWS security platform (WAF, ALB, Kinesis Firehose, S3, Glue, Athena, Lambda, EventBridge, SNS, CloudWatch) provisioned entirely through modular Terraform across multiple environments.
- Built a serverless log-analytics pipeline that ingests AWS WAF logs via Kinesis Firehose into a partitioned S3 data lake, catalogued with AWS Glue and queried via Amazon Athena, powering both scheduled reports and live Grafana log dashboards.
- Implemented an automated reporting system using EventBridge-scheduled Lambda functions that run parameterised Athena queries and deliver HTML/CSV security reports via SNS on daily, weekly, and monthly cadences.
- Stood up a self-hosted observability stack (Prometheus, Grafana, Alertmanager) on EC2 with 9 scrape jobs and 12 PromQL alert rules covering both security events and infrastructure health.
- Authored a Terraform-validated, security-scanned, least-privilege IAM model with per-service roles and customer-managed KMS encryption across S3, SNS, and Firehose.
- Wrote an automated test suite (pytest unit tests, integration tests, and Terraform fmt/validate/plan checks) covering Prometheus configuration, Grafana dashboard structure, Lambda report logic, and Terraform hygiene.

8.3 The 60-Second Verbal Walkthrough (STAR-style)

Situation: Most personal AWS portfolio projects stop at 'I deployed WAF and looked at CloudWatch.' I wanted a project that demonstrated the full lifecycle a real Platform/SRE team owns: prevention, data pipeline, automated reporting, and self-hosted observability.

Task: Design and build a system where AWS WAF protects an application, every security decision is durably logged and queryable, stakeholders get automated reports without anyone running a manual query, and engineers get real-time visibility and alerting — all as Infrastructure as Code with tests.

Action: I attached a WAF Web ACL (managed rule groups + a rate-based rule) to an ALB inside a VPC, routed WAF logs through Kinesis Firehose into a partitioned S3 data lake, catalogued it with Glue, and queried it with Athena. I automated reporting with EventBridge → Lambda → SNS, and built a parallel real-time path: CloudWatch metrics pulled into a self-hosted Prometheus/Grafana/Alertmanager stack on EC2, with 12 alert rules and 5 purpose-built Grafana dashboards. Everything is Terraform-modular and covered by a pytest + Terraform-validation test suite.

Result: A working, testable, fully-IaC platform I can demo end to end — fire a simulated SQLi attack, show the block in the WAF metrics within 15 seconds in Grafana, show the same event land in the Athena-backed log dashboard a little later, and show the scheduled report email that summarises it the next morning.

8.4 Honest Framing — What to Say When Asked About Scale

- This is a single-account sandbox deployment, not a production system handling real production traffic — say so directly if asked.
- The value being demonstrated is architectural breadth and depth across five AWS/OSS skill domains (edge security, data engineering, serverless automation, observability, IaC/security) within one coherent system, not raw scale.
- **For production:** move the EC2 monitoring fleet behind a NAT/private subnet; consider Amazon Managed Service for Prometheus/Grafana for HA; add WAF CloudFront/global scope if the app serves multiple regions; add cross-account log

archival for tamper resistance.

9. Interview Questions & Model Answers

Organised by subsystem, mirroring the service deep-dive. Answers are written in first person, ready to be spoken largely as-is.

9.1 AWS WAF & Security

Q. Walk me through what AWS WAF actually does in your architecture.

AWS WAF sits as a Web ACL attached directly to my ALB. It inspects every incoming HTTP request at layer 7 before the ALB forwards anything. It evaluates the request against AWS Managed Rule Groups for SQL injection and common bad inputs/XSS, plus a custom rate-based rule that blocks any single IP exceeding a request threshold in a rolling 5-minute window. Every decision is logged via the native WAF-to-Firehose log destination.

Q. Why did you choose AWS Managed Rule Groups instead of writing your own WAF rules?

Managed Rule Groups are maintained and updated by AWS's own threat-intelligence research, so the SQLi and common-bad-inputs signatures stay current without me tracking new attack patterns myself. I did add one custom rule — the rate-based rule — because that threshold is specific to my application's expected traffic shape.

Q. What's the difference between a regional WAF Web ACL and a CloudFront/global one, and why did you pick regional?

A regional Web ACL attaches to resources like an ALB or API Gateway within a specific region; a CloudFront-scoped Web ACL is deployed globally at the CDN edge. My deployment is a single-region sandbox with the ALB as the direct entry point, so a regional Web ACL is the correct and simplest fit.

Q. How would you detect and respond to a coordinated, distributed attack — many IPs, low rate each?

A pure rate-based rule per-IP wouldn't catch this since no single IP crosses the threshold. That's why I have a TopAttackerSpike alert that looks at aggregate block-rate patterns in Athena logs (sudden surge in distinct attacking IPs hitting the same rule). In production, I'd look at AWS WAF Bot Control or Fraud Control's ATO prevention rule group.

Q. What's actually inside a WAF log record, and why does that schema matter downstream?

Each WAF log record is JSON containing the action taken (ALLOW/BLOCK/COUNT), the terminating rule that fired, the full request metadata (headers, URI, HTTP method, client IP, country), and a timestamp. That schema is exactly what Glue infers as the waf_logs table — which is why Athena queries can filter on fields like httpRequest.country or terminatingRuleId directly.

9.2 Networking & ALB / VPC

Q. Why use an ALB and not a Network Load Balancer here?

WAF only attaches to layer-7 resources — ALB, CloudFront, or API Gateway. An NLB operates at layer 4 and has no concept of HTTP headers, URIs, or request bodies, so it literally cannot host a Web ACL.

Q. How did you design the VPC and subnet layout, and why?

It's a /16 VPC with public subnets across two Availability Zones. I kept it simple for this sandbox — no NAT Gateway, no private subnets — because every compute resource either needs inbound traffic (ALB) or outbound scraping access (monitoring EC2). For production hardening, I'd move the monitoring fleet into private subnets behind a NAT Gateway or SSM Session Manager.

Q. What does the Blackbox Exporter do, and why is it necessary alongside CloudWatch's own ALB metrics?

The Blackbox Exporter actively sends synthetic HTTP probes to the ALB DNS name and exposes a probe_success metric. CloudWatch's native ALB metrics are derived from real traffic — they tell you about traffic that happened, not whether the ALB is reachable right now. Blackbox gives an independent, active health signal even during a period of zero real traffic.

9.3 Data Pipeline — Firehose, S3, Glue, Athena

Q. Why Kinesis Firehose instead of writing logs directly with a Lambda function subscribed to CloudWatch Logs?

Firehose is a native WAF logging destination, so there's zero custom code or compute needed. Firehose also handles buffering, batching, compression, and retry logic automatically. A Lambda-based shipper would mean reimplementing exactly that behaviour myself, with more moving parts and more failure modes.

Q. Walk me through why you partition the S3 bucket by year/month/day.

Partitioning by date means Athena and Glue can prune which S3 prefixes they scan based on a query's date filter. Since Athena bills by bytes scanned, partition pruning directly reduces both query latency and cost.

Q. What does the Glue crawler actually solve that you couldn't just do with a fixed Athena table definition?

If I hand-wrote the Athena DDL once, every new date partition would need a manual ALTER TABLE ADD PARTITION statement. The Glue crawler runs on a schedule, automatically discovers new partitions, and keeps the Catalog current without anyone touching DDL by hand.

Q. How do you control Athena query cost?

Three things: GZIP compression reduces bytes scanned; date partitioning lets queries scan only the relevant day/month; and the Athena workgroup has a per-query data-scanned limit so a badly written query can't silently rack up a large bill.

Q. If log volume grew 100x, what in this pipeline would change first?

Firehose scales transparently. The first pressure point would be Athena query cost and latency. I'd tighten partition granularity (down to hourly) and consider converting logs from JSON to Parquet via Firehose data-format conversion — Parquet's columnar layout cuts Athena's bytes-scanned dramatically.

9.4 Serverless & Eventing — Lambda, EventBridge, SNS

Q. Why Lambda for the report generator instead of a container or cron job on EC2?

The workload runs a handful of times a day for a short duration — there's no reason to pay for an always-on container or EC2 instance just to wait for a cron tick. Lambda bills per invocation and per millisecond, integrates natively with EventBridge, and removes an entire piece of infrastructure to patch.

Q. Athena queries can take longer than a Lambda execution comfortably allows — how did you handle that?

I call StartQueryExecution to kick off each of the six queries asynchronously, then poll GetQueryExecution until each reports SUCCEEDED, then pull results with GetQueryResults. I don't block on a synchronous query call.

Q. What IAM permissions does the report-generator Lambda actually need, and why does that matter?

Only athena:StartQueryExecution and GetQueryResults, s3:GetObject/PutObject scoped to the specific bucket ARNs, and sns:Publish on the reports topic ARN. If this function were ever compromised through a dependency vulnerability, the blast radius is capped at exactly those three actions on those specific resources.

9.5 Observability — Prometheus, Grafana, CloudWatch, Alertmanager

Q. Why run your own Prometheus/Grafana on EC2 instead of just using CloudWatch?

Two reasons. First, PromQL gives materially richer alerting expressions than CloudWatch Alarms — rate() over a counter, multi-metric boolean combinations, and for-duration conditions are straightforward in PromQL and clumsier in native CloudWatch Alarms. Second, Grafana is the only piece that can show a Prometheus metric panel and an Athena SQL panel on the same dashboard. Running the OSS stack myself also forced me to actually learn Prometheus configuration, retention tuning, and alert-rule authoring hands-on.

Q. Explain the CloudWatch Exporter's role — why can't Prometheus just scrape CloudWatch directly?

Prometheus's scrape model expects targets to expose metrics in Prometheus text exposition format over HTTP; CloudWatch's API is a totally different protocol (GetMetricData calls). The CloudWatch Exporter is the translation layer — it polls the CloudWatch API and re-exposes metrics in Prometheus format on its own port.

Q. What's the difference between what Prometheus does and what Alertmanager does?

Prometheus evaluates alerting rules (PromQL expressions) on its evaluation interval and decides whether a rule is currently firing, then pushes firing alerts to Alertmanager. Alertmanager deduplicates identical alerts, groups related alerts together, applies inhibition rules to suppress lower-priority noise, and routes the grouped alert to the correct receiver. Prometheus decides what's wrong; Alertmanager decides who hears about it and how.

Q. Walk me through the HighBlockRate alert rule end to end.

It's a PromQL expression over `aws_wafv2_blocked_requests_sum`, computing a per-second rate and checking whether it exceeds 10 blocks/sec, with a 'for: 5m' clause so a brief spike doesn't trigger a false alarm. Once firing, Prometheus sends it to Alertmanager, which — because it's marked critical severity — routes it to the higher-urgency receiver, ultimately publishing to the alerts SNS topic for on-call notification.

9.6 IAM, KMS & Security Design

Q. Describe your IAM design philosophy for this project.

Every service boundary gets its own role, scoped to exactly the actions and resource ARNs it needs. The Firehose delivery role can only PutObject to the WAF-logs bucket. The Lambda execution role can only start/read Athena queries, read/write the two specific buckets it touches, and publish to one SNS topic. None of these roles are shared across services, and none use wildcard resource ARNs.

Q. Why customer-managed KMS keys instead of default AWS-managed encryption keys?

Default SSE-S3 or AWS-managed KMS keys encrypt data, but you don't get fine-grained control over exactly which principals can decrypt, and key usage doesn't show up distinctly in CloudTrail. A customer-managed key lets me write an explicit key policy naming exactly which roles may use it, gives me an audit trail of every decrypt call, and gives me a real incident-response lever — I can disable or schedule deletion of the key independently.

Q. If you had to defend this design against a security review, what's the weakest point you'd flag proactively?

The monitoring EC2 fleet's public-subnet placement. For a real production review I'd flag it myself: move those instances into private subnets, remove any public IP, and access Grafana either through SSM Session Manager port forwarding or behind an authenticated ALB listener with its own WAF rule.

9.7 Terraform & Infrastructure as Code

Q. How did you structure your Terraform — walk me through the module breakdown.

Each AWS service area is its own module: vpc, alb, waf, firehose, s3, glue, athena, lambda, monitoring, iam, kms, sns, and cloudwatch. Environments live under terraform/environments/ (dev/test/prod), each composing the same modules with environment-specific variables, so dev and prod share identical module logic and only differ in sizing/thresholds/naming.

Q. Why separate environments as separate directories instead of workspaces?

Separate environment directories with their own state files give complete blast-radius isolation — a terraform apply gone wrong in dev cannot touch prod state, because they're entirely separate state files. Workspaces share a backend configuration in a way that makes me less comfortable for anything touching IAM or security-sensitive resources.

Q. How would you extend this Terraform to support multi-region deployment?

I'd parameterise the region per environment and look carefully at which resources are regional vs. global — WAF Web ACLs and KMS keys are regional; IAM is account-wide. I'd also move the WAF scope from regional to CloudFront if traffic needed to be served globally with a single edge-level Web ACL.

9.8 Scenario / Behavioural & System-Design Questions

Q. Your FirehoseDeliveryFailures alert fires at 2 AM. Walk me through your actual incident response.

First, confirm it's real — check the `aws_firehose_delivery_to_s3_success_average` metric trend in Grafana's Infrastructure dashboard to see whether it's a sustained drop or a brief blip. If sustained, check the Firehose delivery stream's own CloudWatch error logs — commonly an IAM permission issue, an S3 bucket policy change, or a KMS key access problem. Since WAF would still be blocking traffic correctly, the urgency is about restoring log durability, not an active breach.

Q. A stakeholder says the weekly report email never arrived. How do you debug that?

Check it in pipeline order: did the EventBridge rule actually fire? Did the Lambda execute and succeed? If the Lambda succeeded, did its six Athena queries all complete, or did one hang and the function error before reaching the SNS publish step? Finally, did SNS actually publish, and is the recipient's subscription still confirmed?

Q. Why didn't you just use Amazon Managed Grafana and Managed Service for Prometheus?

For a real production system, I'd seriously consider it — it removes patching, HA configuration, and backup concerns. For this project specifically, I chose self-hosted deliberately so I'd actually operate Prometheus's configuration file, tune retention and scrape intervals myself, and hand-author the PromQL alert rules. That hands-on operational experience is exactly what a managed service would abstract away.

Q. What would you do differently if you rebuilt this project today?

Convert the WAF log storage format from raw JSON to Parquet to cut Athena scan costs further; move the monitoring EC2 fleet to private subnets behind SSM Session Manager; and look at writing the Lambda report generator in Go rather than Python — partly for cold-start performance, and partly to reinforce the same Go skill set used elsewhere.

10. Quick-Reference Cheat Sheet

Deployment Commands

```
cd terraform/environments/dev
terraform init && terraform apply
terraform output grafana_url
terraform output alb_dns_name

# Generate test traffic
python scripts/attack_simulation/simulate_sqli.py --url http://$ALB --count 20

# Verify Prometheus
curl http://<monitoring-ip>:9090/api/v1/query?query=up
curl http://<monitoring-ip>:9090/api/v1/query?query=aws_wafv2_blocked_requests_sum
```

Numbers Worth Remembering

Prometheus scrape jobs	9
Prometheus alert rules	12
Grafana dashboards	5
Athena SQL panel targets (log analytics dashboard)	13
Athena report query templates	6
Scrape / rule-evaluation interval	15 seconds
Prometheus TSDB retention	30 days
EC2 instances in monitoring fleet	4 (1 monitoring server + 3 agent nodes)
Report schedules	Daily 06:00 UTC / Weekly Mon 07:00 UTC / Monthly 1st 08:00 UTC
Terraform modules	13 (vpc, alb, waf, firehose, s3, glue, athena, lambda, monitoring, iam, kms, sns, cloudwatch)

Related Documentation

[docs/architecture/HLD.md](#) — High Level Design

[docs/architecture/LLD.md](#) — Low Level Design

[diagrams/aws-architecture.drawio](#) — AWS-icon architecture diagram (companion file — open in diagrams.net)

[docs/guides/grafana-athena-guide.md](#) — Grafana Athena data-source setup

[DEPLOY.md](#) — Copy-paste deployment commands

This guide was prepared as a reference companion to the AWS WAF Security Intelligence & Observability Platform project documentation and architecture diagram, for portfolio presentation and interview preparation purposes.